Gate 1 — Feedback and error

1. What are a reference/setpoint, measured value, feedback and error? 2. Why is feedback usually better than blindly executing a precomputed command?

1.1 Reference/setpoint, measured value, feedback and error

In a control system, the objective is to make a system behave in a desired way. For example, an autonomous ground vehicle may need to maintain a particular speed, follow the centre of a road, or reach a specified position. To achieve this, the controller needs to know both the desired behaviour and the vehicle's actual behaviour.

The four basic terms are:

1. Reference or setpoint

The reference is the desired value that the system is trying to achieve. It acts as the target for the controller.

For example, if an autonomous vehicle is supposed to travel at 10 m/s, its reference speed is 10 m/s. Similarly, if a robot is expected to follow a particular line on the ground, the desired line or position serves as its reference.

2. Measured value

The measured value is the actual value of the system at a particular moment. It is obtained using sensors or estimation methods.

For example, a vehicle's speed sensor might report a speed of 8 m/s when the desired speed is 10 m/s. The measured value tells the controller what is actually happening rather than what should be happening.

3. Feedback

Feedback is the process of returning information about the system's actual output to the controller. The controller uses this information to decide whether a correction is needed.

For example, while riding a bicycle, you continuously observe your position relative to the road. If you start drifting to one side, you adjust the handlebar. Your observation of your current position is effectively feedback.

4. Error

Error is the difference between the desired value and the measured value. A basic control system calculates it as:

$$e(t) = r(t) - y(t)$$

Here, $e(t)$ is the error, $r(t)$ is the reference and $y(t)$ is the measured output at time \(t\).

For example, if the desired vehicle speed is 10 m/s and the measured speed is 8 m/s:

$$e = 10 - 8 = 2 \text{ m/s}$$

The positive error indicates that the vehicle is travelling slower than the desired speed. The controller can use this information to increase the throttle.

The sign and interpretation of an error depend on how the reference and measured quantities are defined.

1.2 Why is feedback usually better than blindly executing a precomputed command?

A precomputed command is a command decided in advance, without necessarily checking the system's actual response. This is called open-loop control when the output is not used to correct the command.

Consider a bicycle travelling along a straight road. If the rider turns the handlebar by a fixed amount and never checks the bicycle's position, the bicycle may drift because of wind, uneven ground or an unexpected obstacle.

A feedback-based approach works differently:

Desired behaviour (reference)

Controller calculates error and chooses a correction

Vehicle or robot moves

Sensors measure the actual result

The measured result is fed back to the controller, and the cycle repeats.

Feedback makes the controller capable of responding to changes and disturbances. If the vehicle begins to deviate from its intended path, the controller can detect the deviation and adjust its steering.

Open-loop control can still be useful when the system is predictable and precise feedback is unnecessary. However, feedback is generally more suitable for autonomous vehicles because real-world conditions continuously change.

Application to autonomous vehicles: Feedback is essential for maintaining speed, correcting steering, following paths and responding to disturbances that were not anticipated when the initial command was planned.

Gate 2 — P controller

1. **Explain proportional control: u = Kp · e. 2. What happens when Kp is increased or decreased? 3. Why can a proportional controller still have steady-state error?**

2.1 What is proportional control?

A proportional controller is one of the simplest control methods. It calculates its output based on the current error between the desired value and the measured value.

Its equation is:

$$\boxed{u(t) = K_p e(t)}$$

Where:

- \(u(t)\) is the control output, such as a steering correction, throttle command or motor input.
- \(e(t)\) is the current error.
- \(K_p\) is the proportional gain, which determines how strongly the controller reacts to the error.

The main idea is simple: a larger error generally produces a stronger correction, while a smaller error produces a weaker correction.

For example, imagine an autonomous vehicle is trying to follow the centre of a lane. If the vehicle is only slightly away from the centre, it may need a small steering correction. If it is far away, a stronger correction may be required.

2.2 What happens when \(K_p\) is increased or decreased?

The value of \(K_p\) strongly affects the behaviour of a proportional controller.

When \(K_p\) is increased:

- The controller reacts more strongly to a given error.
- The system may reach the desired value faster.
- The error may decrease more quickly.
- If the gain is too high, the system may overshoot the target, oscillate or become unstable.

For example, if a robot is drifting away from its desired path, a higher proportional gain may make it turn back more aggressively. However, the robot might turn too far and drift to the opposite side, causing repeated zigzagging.

When \(K_p\) is decreased:

- The controller produces smaller corrections.
- The system may respond more slowly.
- Motion may be gentler, but the error may take longer to decrease.
- If the gain is too low, the system may struggle to follow the desired path accurately.

A suitable gain balances responsiveness and stability. Increasing \(K_p\) indefinitely does not guarantee better performance.

2.3 Why can a proportional controller still have steady-state error?

Steady-state error is the remaining difference between the desired value and the actual value after the system has settled.

Consider a vehicle travelling uphill. Maintaining a constant speed requires the engine to produce enough power to overcome gravity, rolling resistance and air resistance.

A proportional controller generates output according to the current error:

$$u = K_p e$$

If the error becomes exactly zero, the proportional output becomes zero. In a system that requires a nonzero control input to maintain the target, the controller may therefore need a small remaining error to generate the required output.

For example, the vehicle might settle at a speed slightly below its desired speed. That small difference creates enough proportional output to maintain the engine command needed to travel uphill.

This does not mean every P controller always has steady-state error. Its presence depends on the system being controlled and the conditions under which it operates.

One way to address persistent error is to introduce integral action, as used in a PI or PID controller.

Application to autonomous vehicles: Proportional control can help correct lane-position errors, maintain speed and regulate motor movement. It is simple and computationally inexpensive, making it useful as a starting point for control-system design.

Gate 3 — PID Controller

3.1 What is PID control?

PID stands for Proportional–Integral–Derivative. It is a widely used control method that combines three different ways of responding to error.

Its equation is:

$$\boxed{u(t) = K_p e(t) + K_i \int_0^t e\,(\tau)\, d\tau + K_d \frac{de(t)}{dt}}$$

Here:

- \(u(t)\) is the controller's output.
- \(e(t)\) is the current error.
- \(K_p\) is the proportional gain.
- \(K_i\) is the integral gain.
- \(K_d\) is the derivative gain.
- The integral term represents accumulated error over time.
- The derivative term represents how quickly the error is changing.

The equation may look complicated, but its intuition is straightforward. A PID controller considers the current error, the history of the error, and the rate at which the error is changing.

3.2 The role of proportional, integral and derivative actions

P

Proportional action — present error

Proportional action responds to the error that exists right now. A large error produces a strong response, while a small error produces a weaker response.

Example: If a vehicle is far from the centre of its lane, P action commands a stronger correction than it would for a small deviation.

I

Integral action — accumulated error

Integral action accumulates error over time. Even if the error is small, a persistent error gradually builds up in the integral term, allowing the controller to apply additional correction.

Example: If a vehicle consistently travels slightly slower than the desired speed, integral action can increase the throttle command until the persistent speed error is removed, provided the system can achieve the target.

D

Derivative action — rate of change

Derivative action responds to how quickly the error changes. It can provide a damping effect, helping the controller reduce excessive corrections as the system approaches its target.

Example: If a vehicle is approaching the desired path quickly, derivative action can help reduce an overly aggressive correction before the vehicle overshoots.

3.3 How do P, I and D work together?

Imagine an autonomous vehicle trying to maintain a target speed.

1. P action increases or decreases the throttle according to the current speed error.
2. I action builds up a correction if the vehicle remains below the desired speed for a long time.
3. D action reacts to rapid changes in the error and can help reduce overshooting.

Together, these terms can provide a faster and more accurate response while reducing persistent error and excessive oscillation.

However, PID performance depends on tuning the gains. Excessive integral action can cause integral windup, while derivative action can amplify measurement noise. PID is not automatically better in every situation; it must be designed for the particular system.

Application to autonomous vehicles: PID controllers can be used for speed regulation, steering control and other low-level vehicle-control tasks. For example, a path-tracking system may calculate a steering error and use a controller to determine how much the steering should be adjusted.

Gate 4 — Path Tracking

4.1 What is path tracking?

Path tracking is the process of controlling a moving vehicle so that it follows a desired route. The route may be a straight line, a curved road or a path calculated by a path-planning algorithm.

Path planning and path tracking are related but different:

- Path planning determines a route from the start to the goal while considering obstacles and other constraints.

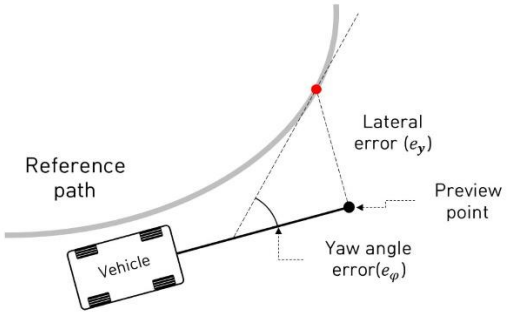
- Path tracking controls the vehicle so that it follows that route in the real world.

For example, an A* algorithm may calculate a route through a grid. The vehicle's path-tracking controller then adjusts its steering to follow the corresponding route as accurately and smoothly as possible.

Two important quantities used in path tracking are cross-track error and heading error.

4.2 What is cross-track error?

Cross-track error (CTE) is the lateral, or sideways, distance between the vehicle's current position and its desired path. It indicates how far the vehicle is from the route it should follow.



mdpi.com

Imagine that a car is supposed to travel along the centre of a road. If it moves towards the right side of the road, the distance between the car's position and the desired centreline represents its cross-track error.

- A small CTE means the vehicle is close to the desired path.
- A large CTE means the vehicle is farther away from the desired path.
- A CTE of zero means the vehicle lies on the reference path at that instant, though it may still be facing the wrong direction.

A path-tracking controller uses this error to determine whether the vehicle needs to steer towards or away from the reference path.

Cross-track error can be represented as a signed or unsigned distance. A signed CTE also indicates which side of the path the vehicle is on, according to the chosen sign convention.

4.3 What is heading error?

Heading error is the angular difference between the direction in which the vehicle is facing and the desired direction of travel along the path. Suppose a car is positioned exactly on the centreline of a curved road but is pointing slightly away from the direction in which the road continues. At that instant, its cross-track error may be zero, but its heading error is not zero.

Heading error is commonly represented as:

$$e_\psi = \psi_{\text{desired}} - \psi_{\text{vehicle}}$$

Where:

- ψ_{desired} is the desired heading angle of the path

- ψ_{vehicle} is the vehicle's actual heading angle.

The difference tells the controller how much the vehicle's orientation differs from the desired direction. In practice, the angle difference must be handled consistently, especially when angles wrap around at \(360^\circ\) or \(2\pi\).

4.4 Why are both position error and heading error important for smooth path following?

Cross-track error and heading error describe two different aspects of the vehicle's motion.

Cross-track error describes position: It tells the controller whether the vehicle is too far to one side of the desired path.

Heading error describes orientation: It tells the controller whether the vehicle is pointing in the correct direction along the path.

If a controller considers only CTE, it may not respond adequately to a vehicle that is on the path but facing away from it. If it considers only heading error, it may fail to correct a vehicle that is pointing in the right direction but is still far from the desired route.

Using both errors helps the vehicle return to the desired path while aligning its orientation with the route. This generally supports smoother steering, reduces unnecessary zigzagging and improves tracking accuracy.

Application to autonomous vehicles: Path-tracking controllers can use CTE and heading error to calculate steering commands. The combination is particularly useful when following curves, changing direction or recovering from deviations caused by disturbances.

Gate 5 —

Introduction to Reinforcement Learning

Reinforcement learning (RL) is a learning approach in which an agent interacts with an environment and improves its decisions using rewards.

- **Agent:** The learner or decision-maker.
- **Environment:** The world in which the agent operates.
- **State:** The current situation of the agent.
- **Action:** A decision the agent can take.
- **Reward:** A numerical signal indicating how desirable an outcome is.
- **Policy:** The strategy used to choose actions based on states.

A **Q-value**, written as (Q(s,a)), estimates the expected long-term reward obtained by taking action (a) in state (s) and then continuing according to a policy. An agent can compare Q-values to identify promising actions.

For example, a robot navigating a maze can learn to prefer actions that lead toward the goal and avoid obstacles by learning from rewards and penalties.